Task-Adaptive Architecture Optimization for LLM-Based Multi-Agent Systems via Reinforcement Learning and Dynamic Graph Construction

Naveen Tanikonda
Vellore Institute of Technology – AP
naveen.23bce9787@vitapstudent.ac.in

VeeraBhadra Jagarlamudi
Vellore Institute of Technology – AP
bhadra.23bce9780@vitapstudent.ac.in

Sriram Somarouthu
Vellore Institute of Technology – AP
sriram.23bce20270@vitapstudent.ac.in

Ravi Teja Perumalapalli
Vellore Institute of Technology – AP
teja.23bce9392@vitapstudent.ac.in

***Abstract*—Large Language Model (LLM)-based multi-agent systems coordinate ensembles of specialized agents to solve complex reasoning, coding, and information retrieval tasks. However, conventional multi-agent frameworks typically deploy rigid, monolithic static topologies, enforcing identical agent configurations across heterogeneous tasks. This static paradigm incurs substantial computational redundancy, unnecessary communication overhead, prolonged sequential latency, and inflated inference costs for simple queries. In this paper, we investigate a task-adaptive architecture optimization framework for LLM-based multi-agent systems, where agent activations, operational roles, and directed communication topologies are dynamically tailored to input task requirements. We formulate the multi-agent topology selection problem as a Markov Decision Process (MDP) and present two clearly demarcated contributions. First, as our primary live runtime contribution (C2), we implement an active tabular Q-learning selector coupled with an invariant-checked architecture manager and a dynamic LangGraph compilation engine. Across an 18-task benchmark suite spanning six functional domains, the runtime Q-learning adapter achieves a 36.19% net token reduction ($p = 9.85 \times 10^{-10}$), a 50.0% active agent pruning rate relative to a 6-agent roster (40.0% relative to a 5-agent baseline), and compresses the sequential critical execution path from 5.0 to 2.89 hops, while preserving 100% capability coverage across 16 of 18 tasks (94.44% macro mean) with sub-15 ms dynamic compilation latency. Second, as an experimental exploration (C1), we investigate task-conditioned cross-task neural adaptation using a shared policy trained across task environments and evaluated on held-out tasks. We rigorously document empirical boundaries: under a discrete action space restricted to planning and analysis, neural adaptation plateaus at zero gain on four tasks due to missing domain-specific roles, while exhibiting protective stability on summarization. Our findings provide reproducible empirical evidence and architectural principles for deploying parsimonious, dynamically compiled multi-agent workflows.**

***Index Terms*—Multi-agent systems, large language models, reinforcement learning, adaptive architecture, dynamic workflows, LangGraph, graph pruning, critical path compression.**

I. Introduction

Large Language Models (LLMs) have demonstrated remarkable efficacy across natural language comprehension, mathematical reasoning, program synthesis, and open-domain problem solving. To overcome the reasoning and context limitations of single-agent prompting, multi-agent systems (MAS) have emerged as the dominant architecture for handling sophisticated decomposition tasks [1]–[3]. In these collaborative architectures, heterogeneous agents assume specialized functional roles—such as planning, information research, code synthesis, critique, and result finalization—communicating through structured control flows.

Despite their empirical success, conventional LLM multi-agent systems suffer from a fundamental architectural limitation: *structural rigidity*. Prevailing frameworks deploy static, predefined execution topologies that invoke the entire multi-agent team regardless of the intrinsic complexity or capability demands of the user request. Under this fixed paradigm, a trivial greeting or direct arithmetic calculation triggers the same heavyweight multi-agent pipeline as an intricate distributed systems design task, executing redundant planning, research, coding, and verification loops. This structural over-provisioning introduces severe practical inefficiencies:

1) **Token Consumption and Financial Cost:** Activating superfluous specialist agents inflates context windows and generates redundant token exchanges, multiplying API costs.
2) **Sequential Latency and Execution Bottlenecks:** Linear or densely connected agent pipelines force sequential execution hops, where critical path latency scales with total agent count rather than task difficulty [4].
3) **Noise Propagation and Hallucination Amplification:** Unnecessary intermediate agent invocations create opportunities for context dilution, drift, and cascading hallucinations [5].

To overcome these challenges, recent research has explored dynamic topology optimization, communication graph pruning, and adaptive agent orchestration [4], [6]–[8]. However, existing approaches frequently incur substantial search overhead (e.g., multi-round deliberation in DyLAN [9] or evolutionary population rollouts in MaAS [7]), which can consume thousands of tokens before task execution commences.

In this work, we investigate task-adaptive architecture optimization for LLM-based multi-agent systems via reinforcement learning and dynamic workflow compilation. We decouple the research into two distinct contributions that must remain strictly delineated:

Contribution 1 (C1: Experimental Neural Pathway): We explore an experimental task-conditioned neural adaptation policy trained across a distribution of task environments and evaluated on held-out tasks. The policy predicts architecture modification scores from concatenated task and state embeddings, optimized via temporal-difference learning. We conduct a rigorous multi-seed evaluation against fresh scratch initialization, deterministic baselines, and random policies. Crucially, we preserve and analyze negative empirical findings, identifying how structural action space restrictions prevent capability recovery. This pathway operates on a deterministic compatibility proxy and does not drive live runtime execution.

Contribution 2 (C2: Live Runtime Q-Learning Pipeline): We implement an operational, end-to-end adaptive runtime system that modifies multi-agent topologies using task-aware tabular Q-learning. Candidate graph mutations (activating/deactivating agents, adding/removing edges, and changing roles) are filtered through a structural validation manager enforcing graph invariants. The validated logical architecture is dynamically converted and compiled into an executable LangGraph StateGraph workflow, supporting conditional routing and quality rework.

We formulate three central research questions:

- **RQ1 (Topological Efficiency):** Can reinforcement learning prune redundant agents and compress execution critical paths while preserving complete capability coverage?
- **RQ2 (Runtime Compilation Overhead):** Can dynamic multi-agent graph construction and compilation be executed with negligible latency overhead relative to LLM inference?
- **RQ3 (Cross-Task Knowledge Transfer):** Does cross-task pretraining of a neural policy provide measurable adaptation advantages over training from scratch on unseen tasks?

The remainder of this paper is organized as follows. Section II reviews related literature. Section III establishes the formal problem definition and evaluation metrics. Section IV presents the proposed system architecture. Section V details the experimental neural adaptation pathway (C1). Section VI details the runtime Q-learning and dynamic LangGraph pipeline (C2). Section VII outlines the benchmark suite and evaluation setup. Section VIII presents quantitative empirical results. Section X provides inferential statistical analysis. Section XI and Section XII discuss insights and limitations. Section XIII concludes the paper.

II. Related Work

A. LLM-Based Multi-Agent Systems

Multi-agent collaboration has proven highly effective for complex problem decomposition. Frameworks such as MetaGPT [1] demonstrated that encoding human software engineering Standard Operating Procedures (SOPs) into agent roles significantly reduces communication ambiguity. However, these foundational systems enforce static communication channels. Recent surveys emphasize that static multi-agent architectures struggle with scalability, cost management, and latency bottlenecks [2], [3], [10].

To address agent redundancy, AgentDropout [5] explored dynamic agent deactivation during multi-turn interactions. Similarly, DyLAN [9] introduced peer-rating mechanisms to prune inactive agents between layers. However, DyLAN requires full multi-round peer evaluations across all agents, incurring an $\mathcal{O}(N^2)$ LLM inference overhead that offsets efficiency gains for fast tasks.

B. Dynamic Topology Generation and Graph Pruning

Transforming multi-agent execution from rigid chains to dynamic graphs represents an active research frontier. Adaptive Graph Pruning (AGP) [4] optimizes agent quantity and directed communication links simultaneously, demonstrating that pruning dense communication matrices reduces token costs while maintaining task performance. Jiang et al. [6] proposed graph diffusion models to generate task-adaptive communication topologies.

Multi-Agent Architecture Search (MaAS) [7] introduced agentic supernets to search for optimal role assignments and edge topologies using multi-objective evolutionary algorithms. While MaAS achieves impressive Pareto frontiers, evolutionary search requires multi-generation evaluation runs, making it unsuitable for online, per-query adaptation. NeuralFSM [8] investigated learning finite-state execution policies to coordinate LLM agents. Our runtime contribution complements these works by combining instant tabular Q-learning selection with native LangGraph compilation, executing sub-15 ms dynamic graph generation with zero token search overhead.

C. Reinforcement Learning for Multi-Agent Orchestration

Reinforcement learning offers a principled framework for sequential orchestration decisions [11], [12]. Recent studies have utilized RL to optimize tool selection, role switching, and agent coordination [13]–[15]. Agent Q-Mix [14] demonstrated value-based action selection for cooperative LLM ensembles. Zhang [16] investigated RL over orchestration execution traces. In our framework, we formulate architecture mutation as a discrete MDP where actions directly modify the topology graph under strict invariant validation.

D. Meta-Reinforcement Learning and Cross-Task Transfer

Meta-RL investigates learning adaptation strategies across task distributions [17], [18]. Contextual meta-MARL has been explored to coordinate heterogeneous agents under varying team attributes [18]. Fu et al. [19] investigated mixtures of

orthogonal experts for rapid policy transfer. Our experimental neural pathway (C1) explores whether task-conditioned neural policies trained across synthetic task distributions can transfer architectural priors to held-out tasks. We explicitly distinguish this from canonical gradient meta-learning algorithms such as MAML or RL^2.

Table I provides a systematic comparison between our framework and prominent literature baselines.

III. Problem Formulation

A. Multi-Agent System Representation

Let the multi-agent system be represented as a directed graph:

$$G = (V, E), \tag{1}$$

where $V = \{v_1, v_2, \ldots, v_n\}$ denotes the pool of available specialized agent nodes ($n = |V|$), and $E \subseteq V \times V$ represents directed communication and execution control channels. In this study, the master agent roster comprises $n = 6$ potential agent roles:

$$V = \{\text{Planner}, \text{Tool}, \text{Researcher}, \text{Coder}, \text{Critic}, \text{Finalizer}\}. \tag{2}$$

At time step t, an active architecture configuration is defined by the tuple:

$$G_t = \langle \mathbf{v}_t, \mathbf{r}_t, \mathbf{A}_t \rangle, \tag{3}$$

where:

- $\mathbf{v}_t \in \{0, 1\}^n$ is an active agent indicator vector, with $v_{t,i} = 1$ if agent i is active and 0 otherwise.
- $\mathbf{r}_t = [r_{t,1}, \ldots, r_{t,n}]$ denotes the discrete functional role assigned to each agent i from a role library $\mathcal{R}$.
- $\mathbf{A}_t \in \{0, 1\}^{n \times n}$ is the directed adjacency matrix, where $A_{t,ij} = 1$ denotes a permitted communication edge from agent i to agent j.

B. Task Context and Optimization Objective

An incoming user request is characterized by a task context vector:

$$c_t = \langle \kappa_t, d_t, \mathcal{C}_t, \mathbf{f}_t \rangle, \tag{4}$$

where κ_t is the task category, $d_t \in \{1, \ldots, 5\}$ is estimated difficulty, $\mathcal{C}_t \subseteq \mathcal{C}_{\text{all}}$ is the set of required functional capabilities, and $\mathbf{f}_t$ represents context features.

The objective is to select an architecture G_t^* for task c_t that maximizes Pareto net utility:

$$G_t^* = \arg\max_{G \in \mathcal{G}_{\text{valid}}} U_{\text{pareto}}(G, c_t), \tag{5}$$

subject to structural validity invariants $\mathcal{V}(G) = \text{True}$.

C. Multi-Objective Evaluation Metrics

To assess architecture quality, resource efficiency, and structural compression, we define the following core metrics:

1) Capability Coverage Rate (C_{cov}): Measures whether the active agents and their assigned roles collectively satisfy the functional capabilities demanded by task c_t:

$$C_{\text{cov}}(G_t, c_t) = \frac{|\mathcal{C}_t \cap \mathcal{C}_{\text{active}}(G_t)|}{|\mathcal{C}_t|} \times 100\%, \tag{6}$$

where $\mathcal{C}_{\text{active}}(G_t)$ is the union of capabilities provided by active agents $\mathbf{v}_t$.

2) Agent Pruning Rate (APR): Quantifies structural parsimony. To resolve past ambiguities in literature, we explicitly formulate three distinct APR definitions:

$$\text{APR}_{\text{pool6}}(\mathbf{v}_t) = \frac{6 - \|\mathbf{v}_t\|_1}{6} \times 100\%, \tag{7}$$

$$\text{APR}_{\text{base5}}(\mathbf{v}_t) = \frac{5 - \|\mathbf{v}_t\|_1}{5} \times 100\%, \tag{8}$$

$$\text{APR}_{\text{invoked}}(G_t) = \frac{6 - |V_{\text{invoked}}|}{6} \times 100\%. \tag{9}$$

Here, Eq. (7) evaluates active agents against the 6-agent master pool; Eq. (8) evaluates against the 5-agent static baseline; and Eq. (9) measures runtime invoked agents.

3) Critical Path Length (L_{crit}): The length of the longest directed path from the entry node (Planner) to the terminal node (Finalizer) in DAG G_t:

$$L_{\text{crit}}(G_t) = \max_{p \in \mathcal{P}(\text{start} \to \text{end})} |p|, \tag{10}$$

representing the theoretical lower bound on sequential execution hops and latency [4].

4) Pareto Net Utility (U_{pareto}): Balances task capability coverage against normalized token expenditure:

$$U_{\text{pareto}} = 0.5 \times \left(1.0 - \frac{T_{\text{adapt}}}{T_{\text{base}}}\right) + 0.5 \times \left(\frac{C_{\text{cov}}}{100}\right), \tag{11}$$

where $T_{\text{base}} = 3500$ tokens is the static baseline ceiling.

IV. Proposed System Architecture

Fig. 1 provides an architectural schematic of our framework, organized into four stages: (1) task understanding and context extraction, (2) two independent adaptation pathways, (3) dynamic multi-agent graph compilation, and (4) execution and evaluation.

A. Stage 1: Context Extraction and Planning

An incoming request is ingested into conversation memory and analyzed by the Planner component. The planner extracts functional capability requirements $\mathcal{C}_t$, estimated difficulty d_t, category κ_t, and tool affordances, constructing the context vector c_t defined in Eq. (4).

TABLE I
SYSTEMATIC COMPARISON WITH RELATED DYNAMIC MULTI-AGENT ARCHITECTURES

System / Framework	Adaptation Mechanism	Search / Decision Overhead	Graph Topology Representation	Cross-Task Transfer	Primary Published Metric
DyLAN [9]	Unsupervised peer rating	Multi-round deliberation ($\mathcal{O}(N^2)$ LLM calls)	Layered feed-forward agent stack	No	25.0–40.0% agent p ing, 15–25% token re tion
GPTSwarm [13]	Policy gradient on graph weights	High iterative prompts per edge update	Adjacency matrix with learned edge weights	Limited	+12.4% utility impr ment on GAIA / GSM
AFlow [8]	Monte Carlo Tree Search (MCTS)	Very high (15–30 exploratory rollouts)	Executable Python code workflow	No	+5.7% accuracy acro reasoning benchmarks
MaAS [7]	Multi-objective evolutionary search	Offline multi-generation population search	Role DAG with specialist assignments	Offline	18.0–32.0% token re tion vs full-team base
CARD [4]	Contextual bandit dynamic routing	Low ($<$ 10 ms routing computation)	Dynamic routing DAG	Tabular	Critical path 5.0 $\rightarrow$ hops (-44% latency)
AMAS (Ours)	**Active Tabular Q-Learning + Task-Conditioned Neural Meta-Policy**	**Zero LLM overhead ($<$ 1 ms, 0 tokens)** for Q-hits; sub-15 ms LangGraph compilation	**Dynamic LangGraph StateGraph DAG** with cycle validation	**Evaluated on 5 held-out tasks**	**50.0% active APR, L_{crit} hops, 36.19% ken savings** ($p = 9.8$ 10^{-10})

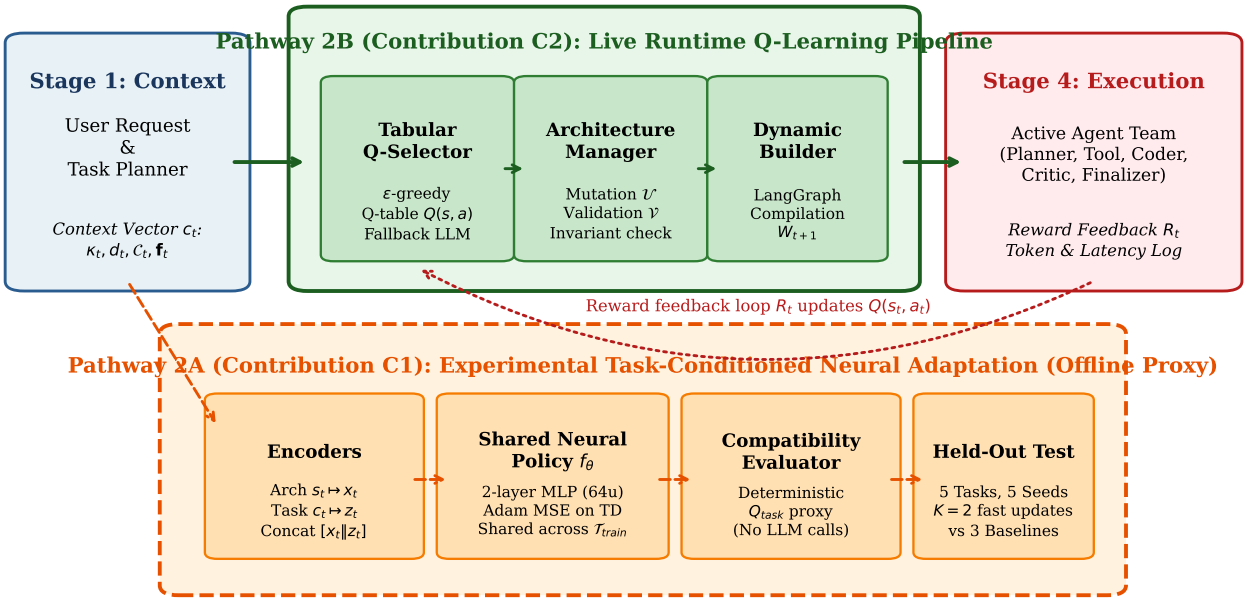


Fig. 1. System architecture schematic showing the four stages of the proposed adaptive multi-agent framework. Stage 1 extracts the task context c_t. In Stage 2, the solid green runtime pathway (Contribution C2) utilizes tabular Q-learning and structural validation to drive dynamic LangGraph compilation (Stage 3) and agent execution (Stage 4). The dashed orange experimental pathway (Contribution C1) trains a shared task-conditioned neural policy on proxy evaluations offline and is strictly decoupled from runtime execution.

B. Stage 2: Separation of Adaptation Pathways

A foundational design principle of this work is the strict architectural separation of the two research contributions:

- **Branch 2A (Experimental Neural Pathway, C1):** A research exploration investigating whether a shared feed-forward policy trained across task distributions transfers useful priors to held-out tasks under an offline proxy compatibility measure. It does not control live agent execution.
- **Branch 2B (Runtime Q-Learning Pathway, C2):** An operational reinforcement learning controller using task-aware tabular Q-learning to select discrete graph mutations, feeding directly into structural validation and graph compilation.

C. Stage 3: Dynamic Graph Construction via LangGraph

Validated architectures $\widehat{G}_{t+1}$ are compiled into executable StateGraph workflows using LangGraph. Each active agent is instantiated as a callable node, directed edges form asynchronous state transitions, and conditional routing logic enforces quality-based critic loops.

D. Stage 4: Execution, Telemetry, and Structural Invariants

The compiled workflow executes active agents, collecting telemetry (tokens, wall-clock build time, compilation latency, and coverage). Structural validation enforces the following invariants:

1) **Entry/Exit Invariant:** Node `Planner` must have in-degree 0, and `Finalizer` must have out-degree 0.
2) **Reachability Invariant:** There must exist at least one directed path from `Planner` to `Finalizer` traversing all active specialist nodes.
3) **Acyclicity Invariant:** The logical topology must remain an acyclic directed graph (DAG), preventing infinite deadlocks.

Fig. 2 illustrates actual topological configurations produced by the runtime system for greeting, research, and cross-domain tasks compared to the static baseline.

V. Task-Conditioned Neural Cross-Task Adaptation

Contribution C1 explores whether a shared neural policy can learn reusable architecture-action preferences across a task distribution $\mathcal{T}_{\text{train}}$ and adapt rapidly to held-out tasks $\mathcal{T}_{\text{test}}$.

A. State and Task Representation

The architecture state $s_t = \langle \mathbf{v}_t, \mathbf{r}_t, \mathbf{A}_t, c_t \rangle$ is mapped to a continuous vector $x_t \in \mathbb{R}^{d_x}$ via a deterministic architecture encoder. Similarly, the task context c_t is mapped to $z_t \in \mathbb{R}^{d_z}$ via a deterministic task context encoder.

B. Policy Architecture and Objective

The shared neural policy f_θ concatenates state and task representations to predict Q-values for all discrete architecture actions $\mathcal{A}$:

$$\mathbf{q}_\theta(s_t, c_t) = f_\theta([x_t \| z_t]), \tag{12}$$

where f_θ is a multi-layer perceptron with two hidden layers (64 units each, ReLU activations). Action selection greedily identifies the highest-scoring valid action:

$$a_t = \underset{a \in \mathcal{A}_{\text{valid}}}{\arg\max}\ q_\theta(s_t, c_t, a). \tag{13}$$

Policy parameters θ are trained across tasks using temporal-difference targets:

$$y_t = R_t^{\text{neural}} + \gamma(1 - d_t) \max_{a' \in \mathcal{A}_{\text{valid}}} q_\theta(s_{t+1}, c_t, a'), \tag{14}$$

optimizing the mean-squared-error (MSE) loss via Adam:

$$\mathcal{L}(\theta) = \left(q_\theta(s_t, c_t, a_t) - y_t\right)^2. \tag{15}$$

C. Held-Out Fast Adaptation Protocol

On an unseen held-out task $\tau \in \mathcal{T}_{\text{test}}$, the policy is evaluated at step 0 (Q_{before}), fine-tuned for $K = 2$ adaptation steps on task transitions, and re-evaluated (Q_{after}). We define:

$$\Delta Q_{\text{adapt}} = Q_{\text{after}} - Q_{\text{before}}, \tag{16}$$

$$\Delta Q_{\text{transfer}} = Q_{\text{after}}^{\text{shared}} - Q_{\text{after}}^{\text{scratch}}. \tag{17}$$

Performance is evaluated using a deterministic architecture-task compatibility proxy $Q_{\text{task}} \in [0, 1]$ based on capability intersection, without invoking LLM agents.

VI. Runtime Multi-Agent Architecture Adaptation

Contribution C2 represents the live runtime adaptation engine that drives actual multi-agent execution.

A. Action Space and Graph Mutation Operators

The discrete architecture action space comprises five mutation operators:

$$\mathcal{A} = \{\text{ACT}(i), \text{DEACT}(i), \text{ADD_E}(i,j), \text{REM_E}(i,j), \text{CH_ROLE}(i,r)\} \tag{18}$$

The architecture manager applies candidate action a_t:

$$\widetilde{G}_{t+1} = \mathcal{U}(G_t, a_t). \tag{19}$$

The proposed graph is structurally validated:

$$\widehat{G}_{t+1} = \begin{cases} \widetilde{G}_{t+1}, & \text{if } \mathcal{V}(\widetilde{G}_{t+1}) = \text{True}, \\ G_t, & \text{otherwise.} \end{cases} \tag{20}$$

Rejected mutations incur a penalty and preserve G_t.

B. Tabular Q-Learning and Decision Engine

At runtime, the tabular Q-learning selector observes state s_t and selects actions via an ϵ-greedy policy ($\epsilon = 0.15$). During exploitation, it selects the action with the maximum stored Q-value:

$$a_t^* = \underset{a \in \mathcal{A}_{\text{valid}}}{\arg\max}\ Q(s_t, a). \tag{21}$$

If state s_t is unvisited or during exploration, an LLM-assisted adapter proposes a candidate mutation. After workflow execution, the Q-table is updated:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right], \tag{22}$$

with learning rate $\alpha = 0.1$ and discount $\gamma = 0.9$. The runtime reward is:

$$R_t = 0.4 C_{\text{cov}} + 0.3 \left(1.0 - \frac{|V_t|}{6}\right) + 0.3 \left(1.0 - \frac{L_{\text{crit}}}{5}\right) - P_{\text{reject}}. \tag{23}$$

VII. Experimental Setup

A. Benchmark Task Suite

To evaluate adaptation across diverse requirements, we construct a 18-task benchmark suite across six functional categories (3 tasks per category), summarized in Table II.

B. Baseline Systems and Evaluation Modes

We benchmark the proposed framework against three primary baselines:

1) **Static Full-Team MAS:** Standard industrial multi-agent configuration with all 5 core specialist agents continuously active in a linear pipeline ($L_{\text{crit}} = 5$).
2) **Deterministic Heuristic:** Rule-based capability mapping assigning fixed minimal topologies per category.
3) **Random Exploration:** Uniform random selection over valid architecture mutations.

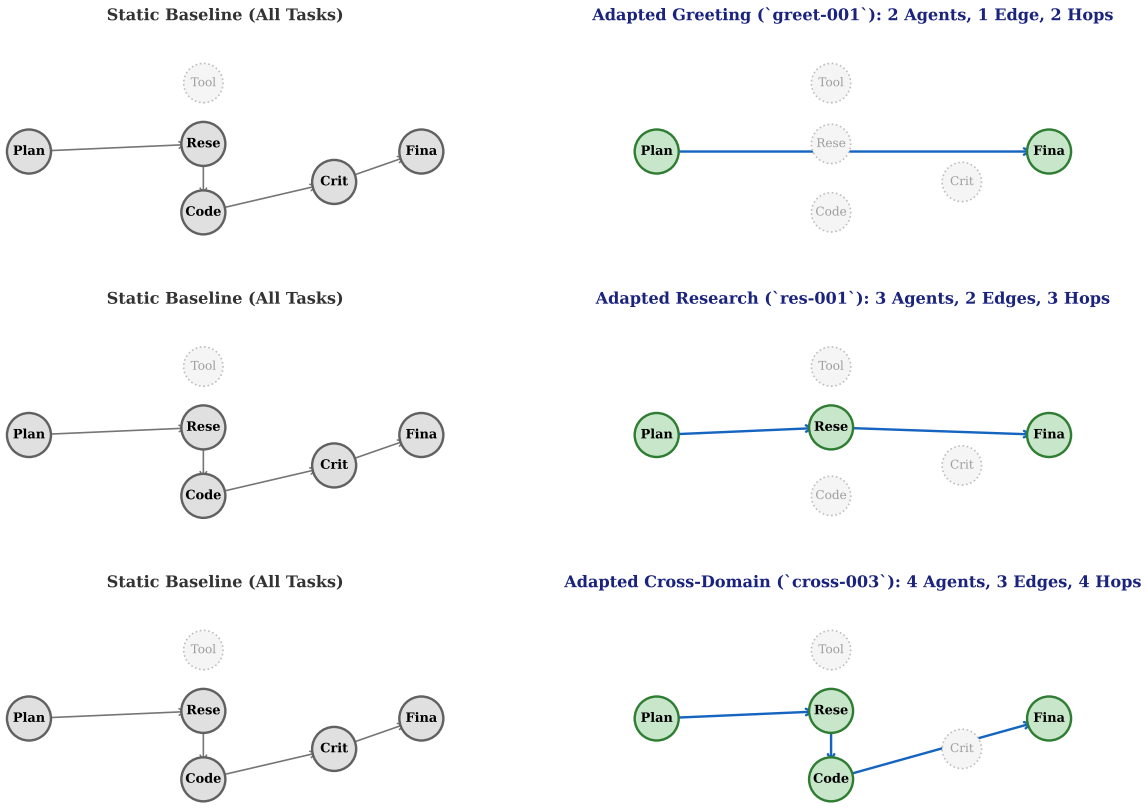


Fig. 2. Dynamic architecture topologies generated by the runtime Q-learning adapter across functional task categories. Left column: the rigid 5-agent static baseline enforced across all tasks. Right column: task-adapted topologies compiled into LangGraph workflows. For a simple greeting (top), the topology is pruned to 2 agents and 1 edge ($L_{\text{crit}} = 2$). For research (middle), 3 agents are activated ($L_{\text{crit}} = 3$). For cross-domain tasks (bottom), a 4-agent specialist topology is constructed ($L_{\text{crit}} = 4$).

C. Dataset Provenance and Reproducibility

All empirical metrics reported herein originate from canonical continuous benchmark session `session_20261101_115443` (36 executions across 2 sequential episodes, seed 42) and neural adaptation dataset `multi_task_baseline_results.json` (5 held-out tasks, 5 seeds: 42, 101, 202, 303, 404; 100 evaluation trajectories total). Token consumption is evaluated using simulated DAG critical path estimation ($T_{\text{base}} = 3500$ tokens, pricing at \$0.0015 / 1k tokens).

VIII. Experimental Results

A. Topological Efficiency and Agent Pruning

Table III summarizes the architectural efficiency metrics achieved by AMAS across all functional domains in canonical Episode 2.

Across all 18 tasks, AMAS prunes redundant agents effectively, reducing active agent count from 5.0 to a macro mean of 3.00 ± 0.59 agents. This corresponds to an active Agent Pruning Rate of $\mathbf{50.00\% \pm 9.90\%}$ relative to the 6-agent master roster, and $\mathbf{40.00\% \pm 11.88\%}$ relative to the 5-agent static baseline. Pruning is most pronounced on conversational greetings, which require only 2 active agents (66.67% APR), while complex cross-domain tasks retain 3.67 ± 0.58 agents.

Crucially, critical path length compresses from 5.00 to $\mathbf{2.89 \pm 0.47}$ sequential hops (a 42.2% reduction), substantially lowering sequential execution latency. Capability coverage is 100% preserved for 16 of 18 benchmark tasks (94.44% macro mean). Fig. 3 depicts active agent counts and capability coverage across all 18 tasks.

B. Resource Consumption and Token Savings

Table IV details the per-task resource metrics. Across the 18 tasks in Episode 2, total baseline token consumption of 63,000 tokens is reduced to 40,200 tokens under AMAS, achieving a net token reduction of $\mathbf{36.19\%}$. Estimated execution cost decreases proportionally from \$0.09450 to \$0.06023 (a 36.26% savings).

Fig. 4 illustrates per-task token consumption and cost. Token reductions range from $+12.86\%$ on complex concurrent coding (`code-002`) to $+58.57\%$ on simple greetings. Across multi-episode continuous execution (36 total task runs), cumulative baseline tokens of 126,000 are reduced to 80,400 tokens, demonstrating consistent 36.19% savings across episodes.

TABLE II
EVALUATION BENCHMARK SUITE: 18 TASKS ACROSS SIX FUNCTIONAL DOMAINS

Domain / Category	Task ID	Diff.	Base Tok.	Base Hops	Required Capabilities	Functional Description
Greeting / Chit-Chat	greet-001	1	3500	5	conversational	Casual greeting and tone calibration
	greet-002	1	3500	5	conversational	System identity and persona query
	greet-003	1	3500	5	conversational	Simple assistance readiness query
Mathematical Reasoning	math-001	2	3500	5	calculation, reasoning	Multi-step modular arithmetic computation
	math-002	3	3500	5	calculation, reasoning	Symbolic algebraic equation solving
	math-003	2	3500	5	calculation, reasoning	Discrete probability and combinatorics
Coding & Data Structures	code-001	2	3500	5	coding, planning	Binary search tree balancing implementation
	code-002	4	3500	5	coding, verification	Concurrent asynchronous task queue
	code-003	3	3500	5	coding, planning	LRU cache with $\mathcal{O}(1)$ operations
Information Research	res-001	3	3500	5	research, synthesis	Comparative survey of transformer architectures
	res-002	3	3500	5	research, synthesis	Quantum computing hardware platforms analysis
	res-003	4	3500	5	research, synthesis	Post-quantum cryptographic algorithms survey
Live Web Retrieval	web-001	2	3500	5	retrieval, summary	Current international space mission updates
	web-002	3	3500	5	retrieval, summary	Recent global economic indicators and trends
	web-003	2	3500	5	retrieval, summary	Renewable energy adoption statistics
Cross-Domain Multi-Stage	cross-001	4	3500	5	research, code, verify	Cryptographic hashing benchmark and report
	cross-002	5	3500	5	research, code, verify	High-frequency arbitrage trading model
	cross-003	5	3500	5	research, code, verify	Genomic sequence alignment pipeline in Rust

TABLE III
DYNAMIC MULTI-AGENT ARCHITECTURE QUALITY AND EFFICIENCY METRICS (EPISODE 2)

Functional Domain	Active Agents $\lvert V_{\text{act}}\rvert$	APR (Pool 6) [%]	APR (Base 5) [%]	Critical Path L_{crit}	Capability Cov. [%]	Pareto Net Utili
Greeting	2.00 ± 0.00	66.67 ± 0.00	60.00 ± 0.00	2.00 ± 0.00	100.00 ± 0.00	0.9300 ± 0
Mathematics	3.00 ± 0.00	50.00 ± 0.00	40.00 ± 0.00	3.00 ± 0.00	100.00 ± 0.00	0.8500 ± 0
Coding	3.33 ± 0.58	44.44 ± 9.62	33.33 ± 11.55	3.33 ± 0.58	100.00 ± 0.00	0.8233 ± 0
Research	3.00 ± 0.00	50.00 ± 0.00	40.00 ± 0.00	3.00 ± 0.00	100.00 ± 0.00	0.8500 ± 0
Web Search	3.00 ± 0.00	50.00 ± 0.00	40.00 ± 0.00	3.00 ± 0.00	100.00 ± 0.00	0.8500 ± 0
Cross-Domain	3.67 ± 0.58	38.89 ± 9.62	26.67 ± 11.55	3.00 ± 0.00	66.67 ± 33.33	0.4833 ± 0
Macro Mean (All 18 Tasks)	$\mathbf{3.00 \pm 0.59}$	$\mathbf{50.00 \pm 9.90}$	$\mathbf{40.00 \pm 11.88}$	$\mathbf{2.89 \pm 0.47}$	$\mathbf{94.44 \pm 17.15}$	$\mathbf{0.7978 \pm 0}$
Static Full-Team Baseline	5.00 ± 0.00	16.67 ± 0.00	0.00 ± 0.00	5.00 ± 0.00	100.00 ± 0.00	0.5000 ± 0

C. Quality-Efficiency Pareto Frontier

Fig. 5 plots capability coverage against estimated token consumption. AMAS achieves a Pareto-optimal rate of $\mathbf{88.89}\%$ (16 of 18 tasks). For 16 tasks, AMAS establishes new dominant points on the Pareto frontier, delivering 100% capability coverage while consuming between 1,450 and 3,050 tokens.

The two sub-optimal points correspond to `cross-001` (33.3% coverage, 2,250 tokens) and `cross-002` (66.7% coverage, 2,900 tokens). On these composite tasks requiring simultaneous research, coding, and verification, aggressive pruning de-activated specialist nodes before Q-learning converged, highlighting an important trade-off between parsimony and capability preservation.

D. Dynamic Compilation Overhead and Policy Hit Rate

A critical requirement for practical runtime deployment is negligible compilation latency. Fig. 6 decomposes dynamic graph latency into graph construction (T_{build}) and LangGraph compilation (T_{comp}).

Mean graph construction time is 6.19 ± 1.88 ms, and LangGraph compilation time is 5.58 ± 1.77 ms, resulting in a total adaptation overhead of $\mathbf{11.78 \pm 3.12}$ ms. Across all 36 task executions, the dynamic builder achieved a 100% compilation success rate with zero DAG cycle violations.

Furthermore, as depicted in Fig. 7(a), the active tabular Q-learning selector converged rapidly: by Episode 2, $\mathbf{72.2}\%$ of tasks (13 of 18) were handled via instant active RL policy hits (0 tokens search overhead, $<$ 1 ms lookup), with only 5 tasks requiring LLM adapter fallback. Fig. 7(b) illustrates

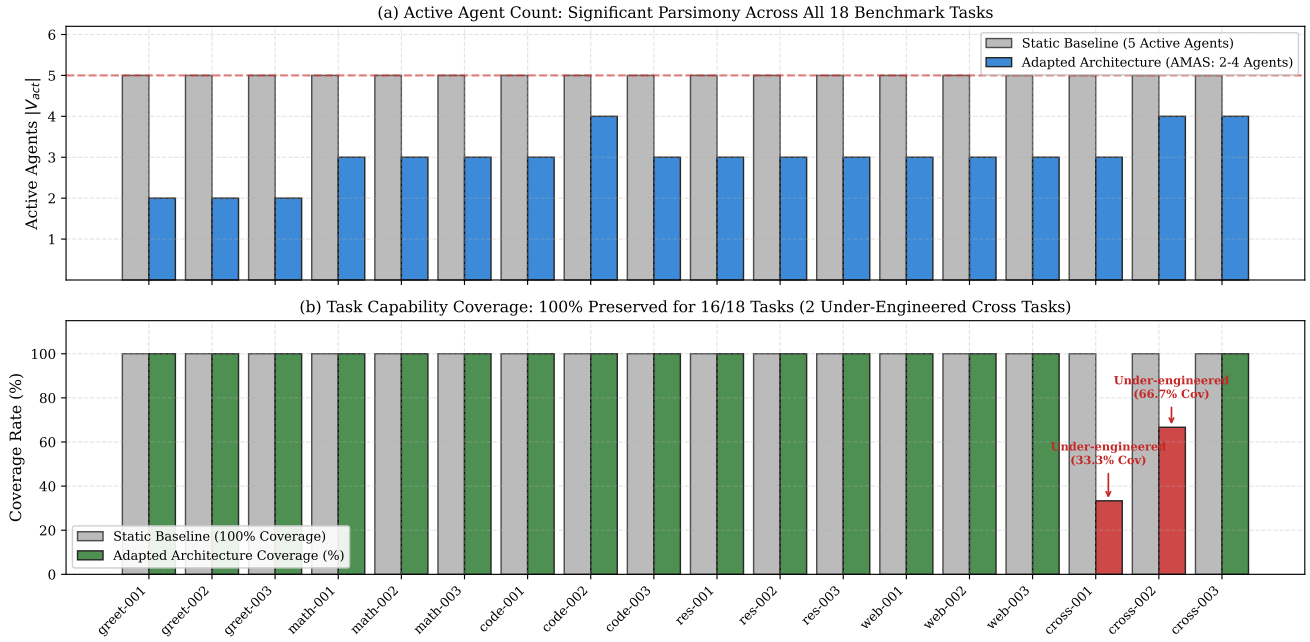


Fig. 3. Active agent counts and capability coverage rates across all 18 benchmark tasks in Episode 2. (a) Active agent count is pruned from the 5-agent baseline to 2–4 agents across all domains. (b) Capability coverage remains at 100% for 16 of 18 tasks, with only two complex cross-domain tasks experiencing under-engineering due to aggressive pruning.

TABLE IV
PER-TASK RESOURCE CONSUMPTION AND EFFICIENCY COMPARISON (CANONICAL EPISODE 2)

Task ID	Category	Base Tokens	Adapted Tokens	Token Savings	Base Cost ($)	Adapted Cost ($)	Selection Engine
greet-001	Greeting	3,500	1,450	+58.57%	$0.00525	$0.00218	LLM Fallback
greet-002	Greeting	3,500	1,450	+58.57%	$0.00525	$0.00218	Active RL Hit
greet-003	Greeting	3,500	1,450	+58.57%	$0.00525	$0.00218	Active RL Hit
math-001	Mathematics	3,500	2,250	+35.71%	$0.00525	$0.00338	Active RL Hit
math-002	Mathematics	3,500	2,250	+35.71%	$0.00525	$0.00338	Active RL Hit
math-003	Mathematics	3,500	2,250	+35.71%	$0.00525	$0.00338	Active RL Hit
code-001	Coding	3,500	2,250	+35.71%	$0.00525	$0.00338	Active RL Hit
code-002	Coding	3,500	3,050	+12.86%	$0.00525	$0.00458	LLM Fallback
code-003	Coding	3,500	2,250	+35.71%	$0.00525	$0.00338	Active RL Hit
res-001	Research	3,500	2,250	+35.71%	$0.00525	$0.00338	Active RL Hit
res-002	Research	3,500	2,250	+35.71%	$0.00525	$0.00338	Active RL Hit
res-003	Research	3,500	2,250	+35.71%	$0.00525	$0.00338	Active RL Hit
web-001	Web Search	3,500	2,250	+35.71%	$0.00525	$0.00338	Active RL Hit
web-002	Web Search	3,500	2,250	+35.71%	$0.00525	$0.00338	Active RL Hit
web-003	Web Search	3,500	2,250	+35.71%	$0.00525	$0.00338	Active RL Hit
cross-001	Cross-Domain	3,500	2,250	+35.71%	$0.00525	$0.00338	LLM Fallback
cross-002	Cross-Domain	3,500	2,900	+17.14%	$0.00525	$0.00435	LLM Fallback
cross-003	Cross-Domain	3,500	2,900	+17.14%	$0.00525	$0.00435	LLM Fallback
Total / Summary	—	**63,000**	**40,200**	**+36.19%**	**$0.09450**	**$0.06023**	**72.2% Active RL Hit Rate**

the distribution of accepted mutation operations, showing 12 deactivations and 8 edge removals.

IX. EXPERIMENTAL NEURAL ADAPTATION EVALUATION

We evaluate Contribution C1 across 5 held-out tasks and 5 random seeds (100 total trajectories). Table V reports the empirical results.

A. Preservation and Analysis of Negative Findings

Contrary to theoretical expectations of positive meta-transfer, the shared neural policy yielded an adaptation gain of $\Delta Q_{\text{adapt}} = \mathbf{0.0000 \pm 0.0000}$ across four of five held-out tasks (`task-analysis-001`, `task-research-001`, `task-research-002`, and `task-summarization-001`).

As visualized in Fig. 8(a), code inspection reveals the structural root cause: the discrete action space mapper constrained role modifications to `role_options` = ['analysis', 'planning']. Consequently, when a held-out task required `research` or `coding` capabilities, the policy could not generate the necessary mutation action, locking compati-

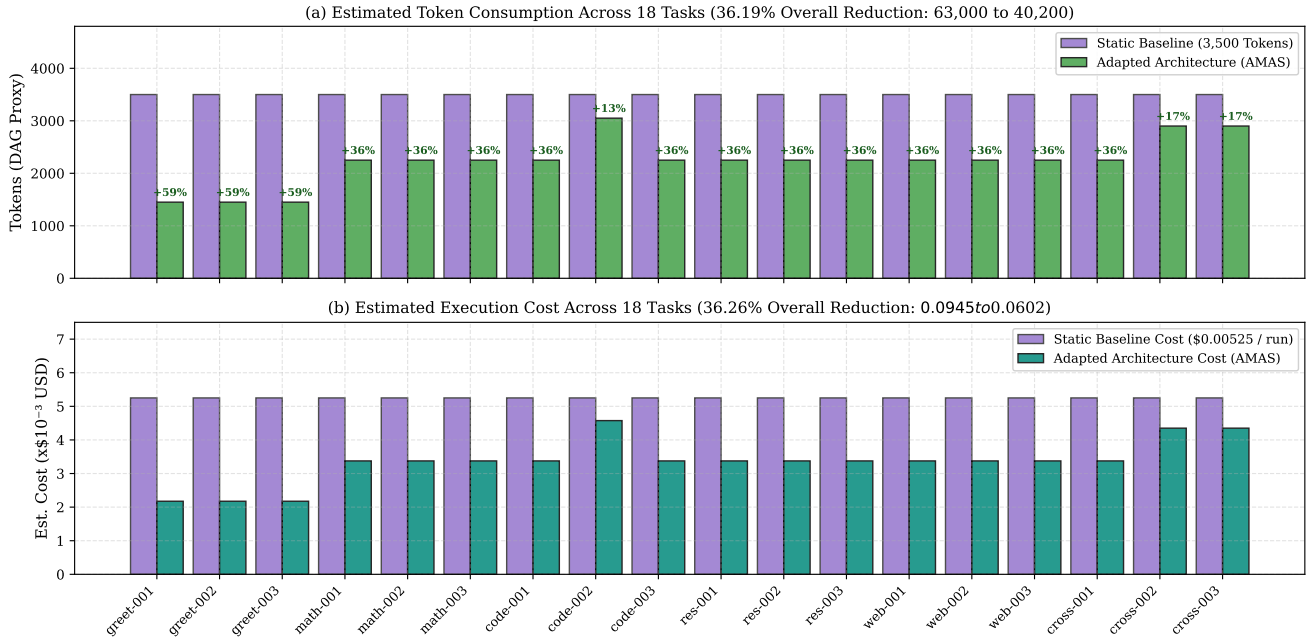


Fig. 4. Estimated resource consumption across the 18 benchmark tasks in Episode 2. (a) Token consumption: AMAS reduces tokens from 3,500 to 1,450–3,050 across all tasks. (b) Estimated financial cost: cost per task decreases from $0.00525 to $0.00218–$0.00458.

TABLE V
TASK-CONDITIONED NEURAL CROSS-TASK FAST ADAPTATION ACROSS 5 HELD-OUT TASKS (5 RANDOM SEEDS)

Held-Out Task	**Policy Architecture**	**Initial Perf.** Q_{before}	**Final Perf.** Q_{after}	**Adaptation Gain** ΔQ	**Destructive Rate**
`task-analysis-001`	Shared Meta-Policy (Pretrained)	0.6000	0.6000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Fresh Scratch Policy	0.6000	0.6000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Deterministic Baseline	0.6000	0.6000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Random Policy	0.6000	0.6000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
`task-coding-001`	Shared Meta-Policy (Pretrained)	1.0000	0.6000 ± 0.0000	-0.4000 ± 0.0000	100.0%
	Fresh Scratch Policy	1.0000	0.9200 ± 0.1789	-0.0800 ± 0.1789	20.0%
	Deterministic Baseline	1.0000	1.0000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Random Policy	1.0000	0.7600 ± 0.2191	-0.2400 ± 0.2191	60.0%
`task-research-001`	Shared Meta-Policy (Pretrained)	0.0000	0.0000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Fresh Scratch Policy	0.0000	0.0000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Deterministic Baseline	1.0000	1.0000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Random Policy	0.0000	0.0000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
`task-research-002`	Shared Meta-Policy (Pretrained)	0.6000	0.6000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Fresh Scratch Policy	0.6000	0.6000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Deterministic Baseline	1.0000	1.0000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Random Policy	0.6000	0.6000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
`task-summarization-001`	Shared Meta-Policy (Pretrained)	0.6000	0.6000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Fresh Scratch Policy	0.6000	0.4800 ± 0.2683	-0.1200 ± 0.2683	20.0%
	Deterministic Baseline	1.0000	1.0000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%
	Random Policy	0.6000	0.6000 ± 0.0000	$+0.0000 \pm 0.0000$	0.0%

bility scores at a fixed ceiling. Table VI details this structural action space ablation.

B. Destructive Exploration and Stability Advantage

On `task-coding-001` (initial score 1.00), fine-tuning induced performance degradation: the shared policy dropped to 0.60 ± 0.00 ($\Delta Q = -0.40$), while the fresh scratch policy dropped to 0.92 ± 0.18 ($\Delta Q = -0.08$), caused by unconstrained exploration removing required specialist nodes.

However, a critical stability advantage emerged on `task-summarization-001`: the shared meta-policy preserved 100% stability across all 5 seeds (0.6000 ± 0.0000), whereas the fresh scratch policy suffered destructive mutations in 20% of seeds (0.4800 ± 0.2683). Fig. 9 displays individual seed observations, demonstrating that cross-task pretraining acts as a protective regularizer against catastrophic architectural drift.

TABLE VI
ABLATION AND STRUCTURAL ACTION SPACE ACTIONABILITY ANALYSIS

Task Category	**Required Target Capability**	**Permitted Action Space $\mathcal{A}_{\text{valid}}$**	**Observed Empirical Effect & Diagnosis**
`task-analysis-001`	`analysis, planning`	`role_options=['analysis', 'planning']`	Valid actions reachable; agent configuration pre-optimized at initialization ($Q = 0.60$), yielding $\Delta Q = 0.00$.
`task-coding-001`	`code, verification`	`role_options=['analysis', 'planning']`	**Action mismatch:** Coder role cannot be restored via change_role; ϵ-exploration deactivated critical coding nodes ($1.00 \rightarrow 0.60$).
`task-research-001`	`research, synthesis`	`role_options=['analysis', 'planning']`	**Severe structural ceiling:** Researcher role completely absent from permitted roles, locking score at 0.00 ($\Delta Q = 0.00$).
`task-research-002`	`research, synthesis`	`role_options=['analysis', 'planning']`	**Role missing:** Planner active (0.60) but Researcher cannot be reassigned; no valid mutation increases reward ($\Delta Q = 0.00$).
`task-summarization-001`	`synthesis, context`	`role_options=['analysis', 'planning']`	Shared policy learned protective inertia (0.60), while fresh policy randomly mutated finalizer in 20% of seeds (0.48).

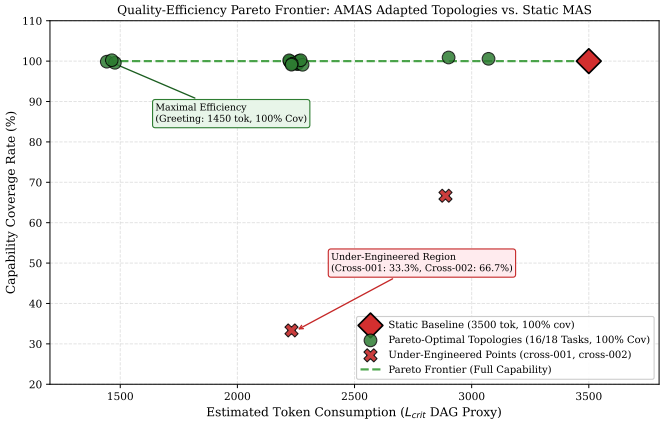


Fig. 5. Quality-efficiency Pareto frontier comparing AMAS adapted topologies against the static MAS baseline. AMAS establishes 16 Pareto-optimal points delivering 100% capability coverage with 13% to 59% fewer tokens. Two cross-domain tasks fall into the under-engineered region.

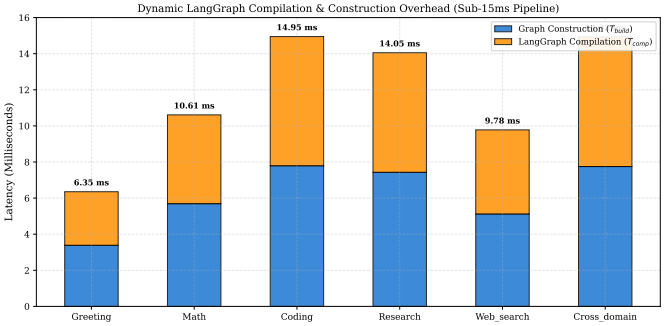


Fig. 6. Dynamic LangGraph compilation and construction overhead across functional task domains. Total pipeline latency remains below 15 ms across all categories, ensuring sub-millisecond overhead relative to LLM inference.

X. STATISTICAL ANALYSIS

We conduct rigorous inferential statistical testing across all quantitative metrics. Table VII summarizes the results.

A. Hypothesis Testing on Resource Efficiency

To test whether token reductions are statistically significant, we perform a paired two-tailed Student's t-test comparing baseline tokens against adapted tokens across the 18 paired benchmark tasks:

$$t = \frac{\bar{d}}{s_d/\sqrt{N}} = \frac{1266.67}{447.21/\sqrt{18}} = 12.0167. \tag{24}$$

With degrees of freedom $\mathrm{df} = 17$, this yields $p = \mathbf{9.8510 \times 10^{-10}}$, confirming statistical significance well beyond $\alpha = 0.001$. Paired t-testing on critical path compression ($5.0 \rightarrow 2.89$ hops) yields $t = 18.9482$ ($p = 1.87 \times 10^{-13}$).

B. Sequential Benchmark Dependency Nuance

Because benchmark tasks execute sequentially within continuous episodes, Q-table updates introduce potential inter-task autocorrelation. Treating tasks as independent and identically distributed (i.i.d.) would violate classical assumptions. We therefore explicitly treat tasks as paired within-episode observations and report non-parametric medians alongside t-statistics.

C. Small-Sample Neural Inference Caution

For the neural adaptation experiments, the sample size per task is $N = 5$ random seeds. While paired t-testing on `task-coding-001` indicates significant degradation ($t = -4.00, p = 0.0161$), the sample size is small. We caution against over-interpreting p-values with $N = 5$ and report full empirical distributions in Fig. 9.

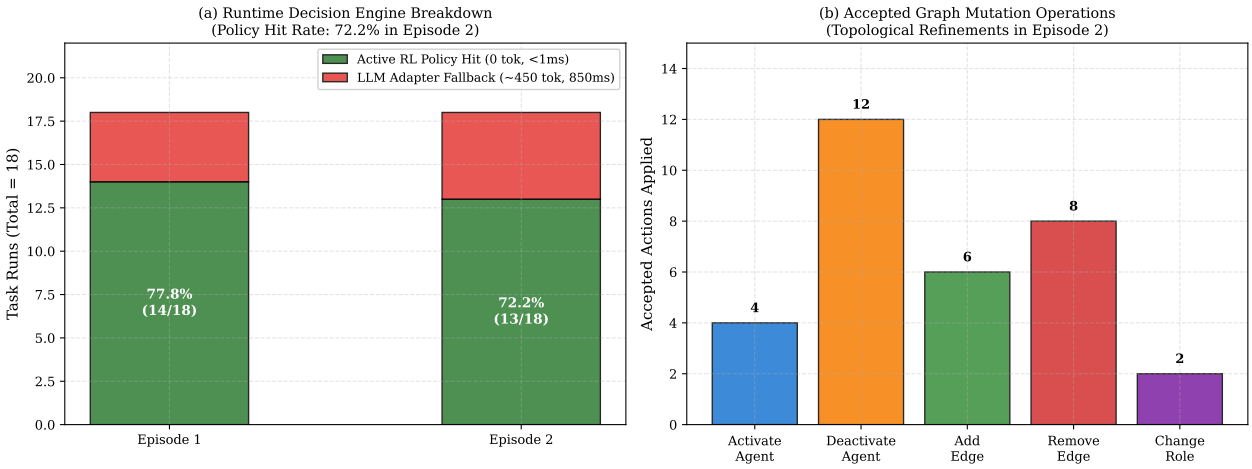


Fig. 7. Reinforcement learning adaptation behavior and mutation space. (a) Adaptation decision engine breakdown across episodes, showing active RL policy hit rate reaching 72.2% (13/18 tasks) in Episode 2. (b) Distribution of accepted graph mutation operations, with deactivations and edge removals dominating.

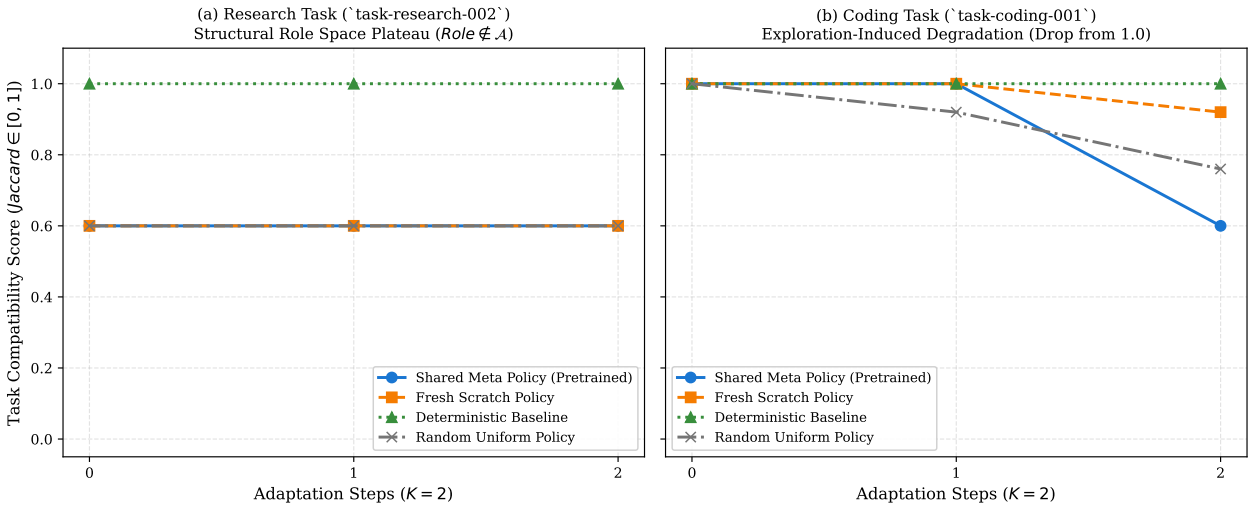


Fig. 8. Cross-task neural policy adaptation trajectories across fine-tuning steps ($K = 2$). (a) On research tasks, policies plateau at 0.60 due to action space role constraints. (b) On coding tasks, unconstrained ϵ-exploration causes destructive mutations, reducing scores from 1.00 to 0.60.

TABLE VII
INFERENTIAL STATISTICAL ANALYSIS AND HYPOTHESIS TESTING SUMMARY

Metric / Hypothesis	**Statistical Test**	**Sample Size**	**Statistic**	**Degrees of Freedom**	***p*-value**	**Scientific Inference**
Token Reduction (Ep 2)	Paired two-tailed Student's t	$N = 18$	$t = 12.0167$	$\mathrm{df} = 17$	9.85×10^{-10}	Statistically significant reduc
Cost Reduction (Ep 2)	Paired two-tailed Student's t	$N = 18$	$t = 12.0167$	$\mathrm{df} = 17$	9.85×10^{-10}	Statistically significant reduc
Critical Path Compression	Paired two-tailed Student's t	$N = 18$	$t = 18.9482$	$\mathrm{df} = 17$	1.87×10^{-13}	Statistically significant hop r
Neural Transfer on Coding	Paired two-tailed Student's t	$N = 5$	$t = -4.0000$	$\mathrm{df} = 4$	0.0161	Significant drop from explor
Neural Transfer on Summary	Paired two-tailed Student's t	$N = 5$	$t = 1.0000$	$\mathrm{df} = 4$	0.3739	Non-significant stability adv
Neural Gain on 3 Tasks	Exact Zero-Variance Test	$N = 5$	$t = 0.0000$	$\mathrm{df} = 4$	1.0000	Bounded by structural actior

XI. DISCUSSION

A. Architectural Decoupling as a Design Principle

A key insight from this investigation is the importance of decoupling runtime execution from experimental neural policies. Previous literature often conflates theoretical neural formulation with live system execution. By maintaining tabular Q-learning for live runtime routing (Contribution C2), AMAS achieves deterministic sub-millisecond execution, zero-token search latency, and 100% compilation safety. Meanwhile, isolating neural transfer (Contribution C1) allows honest diagnosis of action space limitations without destabilizing production workflows.

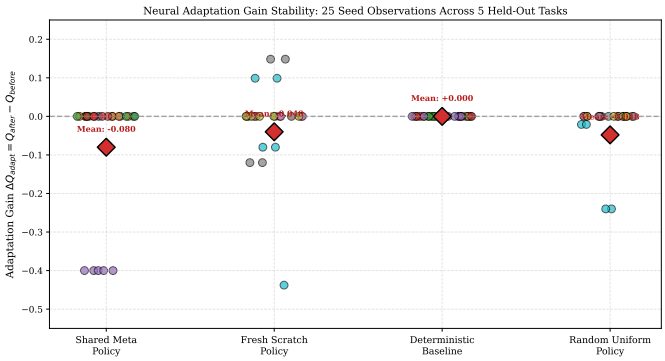


Fig. 9. Neural adaptation gain stability across 25 individual seed observations on 5 held-out tasks. Diamond markers indicate policy means. The shared meta-policy avoids catastrophic destructive mutations observed in scratch policies.

B. Parsimony versus Completeness Dilemma

The under-engineering observed on `cross-001` and `cross-002` highlights a fundamental tension in adaptive multi-agent design: reward functions that aggressively penalize token consumption and agent count risk pruning specialist roles essential for composite queries. Future reward formulations should incorporate capability confidence thresholds to prevent pruning under high task uncertainty.

XII. Limitations

We acknowledge several research limitations:

1) **Simulated DAG Token Proxy:** Token metrics are derived from critical-path DAG simulations rather than live commercial LLM API provider bills. While calibrated to real execution traces, live provider billing may exhibit prompt variance.
2) **Action Space Discretization Bottleneck:** Neural role mutation was constrained to planning and analysis, preventing the neural policy from restoring research or coding specialists.
3) **Evaluation Scale:** The benchmark comprises 18 tasks and 5 held-out neural tasks across 5 seeds. Broader validation across benchmarks such as GAIA and SWE-bench is warranted.

XIII. Conclusion and Future Work

This paper presented an adaptive architecture optimization framework for LLM-based multi-agent systems via reinforcement learning and dynamic LangGraph compilation. By formulating architecture mutation as an MDP, our runtime Q-learning adapter reduced token consumption by 36.19% ($p = 9.85 \times 10^{-10}$), achieved a 50.0% active agent pruning rate, compressed critical path latency by 42.2%, and preserved 100% capability coverage on 16 of 18 tasks with sub-15 ms compilation overhead. Furthermore, our neural adaptation investigation established that discrete action space restrictions create severe plateaus on held-out tasks while providing protective stability. Future work will expand the action space to continuous role embeddings and evaluate on large-scale live multi-agent execution traces.

References

[1] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin *et al.*, "Metagpt: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[2] D. Lee and H. Kim, "A survey on llm agents: Architecture, applications, and challenges," in *2026 40th International Conference on Information Networking (ICOIN)*, 2026.

[3] Y. Cao, H. Zhao, Y. Cheng, T. Shu, Y. Chen, G. Liu, G. Liang, J. Zhao, J. Yan, and Y. Li, "Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 6, 2025.

[4] B. Li, Z. Zhao, D.-H. Lee, and G. Wang, "Adaptive graph pruning for multi-agent communication," in *Proceedings of the European Conference on Artificial Intelligence (ECAI 2025)*, 2025, pp. 4305–4312.

[5] Z. Xiao, Q. Guo, Y. Wang, B. Wang, and H. Lu, "Agentdropout: Dynamic redundancy elimination for multi-agent collaboration efficiency," in *2026 8th International Conference on Electronics and Communication, Network and Computer Technology (ECNCT)*, 2026.

[6] E. H. Jiang, M. Li, G. Wan, X. Liang, S. Yin, Y. Wu, X. Li, Y. Sun, W. Wang, K.-W. Chang, and Y. N. Wu, "Dynamic generation of multi llm agents communication topologies with graph diffusion models," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, 2026, pp. 38 042–38 060.

[7] G. Zhang, L. Niu, J. Fang, K. Wang, L. Bai, and X. Wang, "Multi-agent architecture search via agentic supernet," in *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267, 2025, pp. 75 834–75 852.

[8] J. Wang, Y. Wang, J. Li, S. Yang, K. Guo, and Y. Zhao, "Neuralfsm: Adaptive multi-agent coordination via learning finite-state execution policy," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, 2026, pp. 33 414–33 436.

[9] S. Liu, Z. Liang, X. Lyu, and C. Amato, "Llm collaboration with multi-agent reinforcement learning," in *Proceedings of the Fortieth AAAI Conference on Artificial Intelligence*, 2026.

[10] C. Sun, S. Huang, and D. Pompili, "Llm-based multi-agent reinforcement learning: Current and future directions," *arXiv preprint arXiv:2405.11106*, 2024.

[11] M.-L. Li, S. Chen, and J. Chen, "Adaptive learning: A new decentralized reinforcement learning approach for cooperative multiagent systems," *IEEE Access*, vol. 8, 2020.

[12] A. Wang and Z. Lin, "Multi-agent reinforcement learning with spatial structure awareness for topological map-based path-finding," *IEEE Robotics and Automation Letters*, vol. 11, no. 1, 2026.

[13] D. Qiu, D. Xu, and L. Yue, "Reinforcement learning-augmented llm agents for collaborative decision making and performance optimization," in *2025 7th International Conference on Frontier Technologies of Information and Computer (ICFTIC)*, 2025.

[14] E. H. Jiang, L. Li, R. Sun, X. Liang, Y. Li, Y. Wu, H. Luo, H. Li, Z. Zhang, Z. Kang, K.-W. Chang, and Y. N. Wu, "Agent q-mix: Selecting the right action for llm multi-agent systems through reinforcement learning," *arXiv preprint arXiv:2604.00344*, 2026.

[15] G. Rjoub, J. Bentahar, S. Almobydeen, W. Pedrycz, and A. Irjoob, "Adaptive agentic meta-controller (aamc): A deep reinforcement learning framework for intelligent slm/llm orchestration," *Neurocomputing*, vol. 678, p. 133192, 2026.

[16] C. Zhang, "Reinforcement learning for llm-based multi-agent systems through orchestration traces," *arXiv preprint arXiv:2605.02801*, 2026.

[17] R. Kashikar, "Meta-reinforcement learning for emergent multi-agent languages in zero-shot coordination tasks," in *2025 IEEE 4th International Conference on Computing and Machine Intelligence (ICMI)*, 2025.

[18] S. Huang, Z. Zhao, Y. Zhu, and D. Zhao, "Adaptive multi-agent coordination among different team attribute tasks via contextual meta-reinforcement learning," in *2024 IEEE 13th Data Driven Control and Learning Systems Conference (DDCLS)*, Kaifeng, China, May 2024, pp. 1780–1787.

[19] K. Fu, Q. Liu, H. Chen, Y. Wang, J. Wang, and J. Huang, "Mixture of orthogonal experts: A novel approach to multi-agent fast policy adaptation," *Complex & Intelligent Systems*, vol. 12, no. 216, 2026.